

Resources

Nice — I'll outline this as if I'm handing a single design/engineering spec to your backend team. I'll include everything: features → sub-features → required APIs → example request/response JSON → data model / validations → background jobs, security, scaling notes, and recommended improvements. You can hand this directly to a backend developer.

I'll group by feature, then give DB schema fragments, then API contracts, then infra & ops suggestions.

CleanLink — Backend & API Specification

Audience: backend engineers, API developers, product managers

Assumptions: REST JSON API (HTTPS), JSON Web Tokens (JWT) or Firebase-style Auth, object IDs are UUID v4 (or 24-hex Mongo IDs), timestamps in ISO 8601 UTC. Images use signed uploads (presigned S3/GCS URLs). Use pagination via `page / limit` or cursor (`cursor`).

Table of contents (quick)

1. Features overview
 2. Data model (primary tables/collections & fields)
 3. API endpoints — grouped by feature with request/response examples and validations
 4. Background jobs, AI & image pipeline, duplication detection
 5. Security, rate limits, abuse prevention, privacy constraints
 6. Monitoring, metrics, and operational recommendations
 7. Additional recommended improvements & roadmap items
-

1) Complete list of features (top-level)

1. Authentication & Authorization (Email/Password, Google OAuth, Phone OTP)
 2. User Profile & Civic Sense System (points, levels, badges)
 3. Region & Personalization (Country/State/City preferences)
 4. Reports (create, read, update status, delete)
 5. Media Handling (images: upload, thumbnails, after-clean photos)
 6. Voting (upvote/downvote), Comments, and Basic Moderation
 7. MCD/Admin dashboard & actions (verify/assign/resolve)
 8. AI Verification & Duplicate Detection (legitimacy score, duplicate suggestions)
 9. Trending / Popular Problems / Search / Filters
 10. Rewards & Redemption (points → perks)
 11. Notifications (push/email/in-app)
 12. Audit logs & appeal / ban flow
 13. Reporting & Analytics (hotspots, resolution metrics)
 14. Offline support / queued submissions (client-side; backend API supports idempotency)
 15. Rate-limiting, abuse detection, escalations
-

2) Data model (simplified, collections/tables)

Use whichever DB is preferred — below uses neutral fields suitable for SQL or document DB (Mongo/Postgres).

2.1 Users

```
User {  
  id: UUID,  
  username: string,    // optional, unique when present  
  phone: string,       // normalized E.164, unique optional  
  email: string,       // lowercase, unique optional  
  password_hash: string, // salted, only if email/password
```

```

auth_providers: [ { provider: 'google'|'phone'|'email', provider_id } ],
avatar_url: string|null,
region: { country, state, city, area_id? }, // user-preferred for feed
is_verified: boolean, // optional Aadhaar/phone verified
civic_points: integer,
civic_level: integer,
badges: [string],
trust_score: float, // hidden model value for abuse detection
created_at: timestamp,
updated_at: timestamp,
status: 'active'|'suspended'|'banned'
}

```

Indexes: email, phone, username, region.city, trust_score.

2.2 Reports

```

Report {
  id: UUID,
  reporter_id: UUID|null, // null if anonymous
  reporter_display: string, // e.g. "Anonymous" or username
  title: string,
  description: string,
  category: 'garbage'|'road'|'water'|'tree'|'electricity'|'other',
  images: [ { url, thumb_url, uploaded_at } ],
  location: { lat: float, lng: float, area_name: string, public_area: string }, // public_area = ward/sector string
  visibility: 'public'|'masked', // public shows area only, exact stored but only accessible to MCD/admin
  ai_score: { legit: float, duplicate_of: [report_id], severity: float },
  upvotes: integer,
  downvotes: integer,
  community_score: float, // normalized combined score
  status: 'pending'|'community_verified'|'mcd_viewed'|'assigned'|'resolved'|'re

```

```
jected'|'duplicate'|'flagged',
  mcd_verified_by: UUID|null,
  mcd_resolution: { cleaned_image_url, notes, resolved_at },
  duplicate_of: UUID|null,
  flags: [ { user_id, reason, created_at } ],
  is_featured: boolean, // for trending/admin pin
  created_at, updated_at
}
```

Indexes: location coordinates (geospatial), category, status, created_at, upvotes.

2.3 Comments

```
Comment {
  id: UUID,
  report_id: UUID,
  author_id: UUID|null,
  text: string,
  created_at, updated_at,
  parent_comment_id: UUID|null // support single-level replies if desired
}
```

2.4 Votes

```
Vote {
  id: UUID,
  report_id: UUID,
  user_id: UUID,
  value: 1 | -1,
  created_at
}
```

Unique composite index: (report_id, user_id).

2.5 Rewards / Redemptions

```
Reward {  
  id: UUID,  
  key: string, // e.g., SOLAR_10P_SUBSIDY  
  title: string,  
  description: string,  
  required_points: integer,  
  available_from, available_until,  
  max_per_user: integer,  
  metadata: { vendor_info? }  
}
```

```
Redemption {  
  id: UUID,  
  user_id: UUID,  
  reward_id: UUID,  
  status: 'requested'|'approved'|'rejected'|'fulfilled',  
  request_data: {...}, // phone/address proof  
  created_at, updated_at  
}
```

2.6 Admin / MCD Entities

```
Admin {  
  id, user_id, region_assigned, role: 'viewer'|'operator'|'manager', api_key?  
}
```

2.7 AuditLog

```
AuditLog {  
  id, actor_id (user/admin/system), action_type, target_type, target_id, details JSON, created_at  
}
```

3) API Endpoints (grouped by feature)

I'll present each feature with endpoints, JSON examples, validations and responses.

Feature A — Authentication & Authorization

1) POST `/api/v1/auth/signup`

Purpose: Register new user (email/password or phone via OTP sign-up flow).

Request:

```
{  
  "username": "raksh",  
  "email": "raksh@example.com",  
  "password": "P@ssw0rd!",  
  "phone": "+919812345678",  
  "region": { "country": "India", "state": "Delhi", "city": "New Delhi" }  
}
```

Validations:

- `email` : valid email format if supplied.
- `password` : min 8 chars, at least 1 digit and 1 uppercase (policy).
- `phone` : E.164 if provided.
- `username` : 3-30 chars, allowed chars alnum + underscore; unique.

Response 201:

```
{
  "user": { "id":"uuid", "username":"raksh","email":"raksh@example.com","region":{"...},"civic_points":0 },
  "token": "JWT....",
  "refresh_token": "..."
}
```

Errors: 400 (validation), 409 (email/phone/username exists).

2) POST `/api/v1/auth/login` (email/password)

Request:

```
{ "email":"raksh@example.com", "password":"P@ssw0rd!" }
```

Response 200:

```
{ "user": {...}, "token":"JWT...", "refresh_token":"..." }
```

Errors: 401 unauthorized.

3) POST `/api/v1/auth/otp/request`

Purpose: request OTP for phone login

Request:

```
{ "phone":"+919812345678" }
```

Response 200:

```
{ "otp_session_id":"abc123", "retry_after_seconds":60 }
```

4) POST `/api/v1/auth/otp/verify`

Request:

```
{ "otp_session_id":"abc123", "otp":"123456" }
```

Response 200:

```
{ "user": {...}, "token":"JWT...", "refresh_token":"..." }
```

5) POST `/api/v1/auth/oauth/google`

Purpose: Accept Google OAuth token, exchange for our JWT

Request:

```
{ "id_token":"google-id-token" }
```

Response 200: same as login.

Notes: Tokens should be JWT signed, include `sub` as user id, expiry short (1h), include refresh token flow.

Feature B — User Profile & Civic Sense

GET `/api/v1/users/me`

Auth required.

Response 200:


```
{
  "id":"uuid",
  "username":"raksh",
  "region":{"country":"India","state":"Delhi","city":"New Delhi"},
  "civic_points": 620,
  "civic_level": 3,
  "badges":["bronze_reporter","eco_champ"],
  "trust_score": 0.82
}
```

PATCH `/api/v1/users/me`

Request (example):

```
{ "username":"newName", "bio":"I report my area", "avatar_url":"https://..." }
```

Validations: username unique, avatar_url valid URL.

Response: updated user object.

GET `/api/v1/users/:id/public`

Public view: returns non-sensitive fields: username (or "Anonymous"), badges, civic_points (optional), region.area_name optional depending privacy.

Feature C — Region & Personalization

GET `/api/v1/regions`

Return list of countries/states/cities (or use geodata microservice).

Response:

```
{ "countries":[ { "name":"India", "states":[ { "name":"Delhi", "cities":[ "New De
lhi", "South Delhi" ] } ] } ] }
```

PATCH `/api/v1/users/me/region`

Request:

```
{ "region": { "country": "India", "state": "Delhi", "city": "South Delhi" } }
```

Response: updated user.

Feature D — Reports (core)

GET `/api/v1/reports` (feed)

Query params: `region` (city), `category`, `status`, `sort` (new|hot|top), `cursor` or `page`, `limit`.

Response 200:

```
{
  "data": [
    {
      "id": "report-uuid",
      "title": "Garbage heap near school",
      "description_preview": "Large pile near XX school, foul smell...",
      "category": "garbage",
      "images": [{"thumb_url": "..."}],
      "area_name": "Rohini Sector-11",
      "lat": 28.72,
      "lng": 77.12, // only if auth and admin/MCD; public may not get exact coord
      "upvotes": 142,
      "comments_count": 12,
      "status": "pending",
      "created_at": "2025-11-15T10:00:00Z"
    }
  ],
}
```

```
"paging": { "next_cursor": "..." }
}
```

Validations: Accept region existence.

POST `/api/v1/reports` (create report)

Auth optional (allow anonymous with rate limits). Clients must send image(s) via presigned upload or multipart.

Request (JSON body):

```
{
  "title": "Garbage pile near Park",
  "description": "Large pile near main gate of Mayur Park, emitting smell. Land mark: bus stop 42.",
  "category": "garbage",
  "images": [ "https://cdn.s3/presigned-upload-key-1" ],
  "location": { "lat": 28.72, "lng": 77.12, "area_name": "Mayur Park, Rohini", "visibility": "masked" },
  "anonymous": true,
  "client_idempotency_key": "uuid-or-string"
}
```

Server-side validations:

- `title` not empty, ≤ 200 chars
- `description` ≤ 1000 chars
- `category` in allowed enums
- `images` non-empty (if category requires)
- `location.lat/lng` in valid ranges
- idempotency by `client_idempotency_key`

Processing:

1. Store report (status `pending`) with reporter_id (or null)
2. Enqueue AI verification job
3. Return created object.

Response 201:

```
{
  "id":"report-uuid",
  "status":"pending",
  "ai_check":"queued",
  "created_at": "...",
  "points_awarded": 0
}
```

Errors: 400 validation, 429 rate limit.

GET `/api/v1/reports/:id`

Response 200:

Full report with allowed fields depending on requester (public vs MCD/admin).

Public view example:

```
{
  "id":"report-uuid",
  "title":"Garbage pile near Park",
  "description":"Large pile ...",
  "category":"garbage",
  "images":[ { "url":"...", "thumb":"..." } ],
  "area_name":"Mayur Park, Rohini",
  "upvotes":142, "downvotes":3, "comments_count":12,
  "status":"pending",
  "ai_score": { "legit":0.89, "severity":0.6 }, // optionally visible
  "timeline": [ { "type":"posted", "at":"..." } ]
}
```

```
}
```

Admin view includes exact lat/lng, reporter contact info.

PATCH `/api/v1/reports/:id` (update)

Use cases: reporter edits description before community verification, admin resolves, MCD adds cleaned image.

Examples:

- Reporter edits: allow if status in `pending`.
- Admin resolves:

```
PATCH /api/v1/reports/uuid
{
  "status":"resolved",
  "mcd_resolution": { "cleaned_image_url":"https://s3/after.jpg", "notes":"Collected by Team 12", "resolved_at":"2025-11-17T09:30:00Z" },
  "mcd_verified_by": "admin-uuid"
}
```

Processing:

- Award civic points to reporter (if valid and not anonymous as configured).
- Create AuditLog.
- Notify user and commenters.

Response: updated report.

POST `/api/v1/reports/:id/flag`

User flags a report as fake/spam.

Request:

```
{ "reason":"duplicate", "details":"This was already posted earlier." }
```

Response 200: flag accepted, triggers moderation workflows.

Feature E — Voting & Comments

POST `/api/v1/reports/:id/vote`

Auth required.

Request:

```
{ "value": 1 } // 1 or -1
```

Validations:

- Value $\in \{1, -1\}$
- One vote per user per report (upsert).
- Rate-limit per minute.

Response:

```
{ "report_id":"...", "score": 123 }
```

POST `/api/v1/reports/:id/comments`

Auth required (or allow guest comments with moderation).

Request:

```
{ "text":"Yes I saw this too", "parent_comment_id": null }
```

Validations: $1 \leq \text{len} \leq 1000$, anti-spam checks.

Response: created comment object.

GET `/api/v1/reports/:id/comments?limit=20&cursor=...` returns paged comments.

Feature F — Admin / MCD Endpoints

Auth: Admin token with region scope.

GET `/api/v1/admin/reports?region=...&status=pending`

List reports for assigned region.

PATCH `/api/v1/admin/reports/:id/assign`

Assign team or worker:

Request:

```
{ "assigned_to": "worker-uuid", "notes": "Urgent clean" }
```

PATCH `/api/v1/admin/reports/:id/resolve`

Same as report update example earlier. Important to include `cleaned_image_url`.

On resolve:

- Award points to reporter: add `civic_points += points_for_resolution` transactionally.
 - Increase reporter trust_score positively.
 - Insert AuditLog record.
-

Feature G — AI Verification & Duplicate Detection (service endpoints)

These are internal endpoints (protected) for AI pipeline to update report statuses.

POST `/internal/ai/reports/:id/result`

Request (from AI worker):

```
{
  "id": "report-uuid",
  "ai_score": { "legit": 0.92, "severity": 0.68, "duplicate_prob": 0.03 },
  "duplicate_of": null,
  "insights": [ "contains_organic_waste", "soiling_level_high" ]
}
```

Server processing:

- Update `report.ai_score`, `report.status` if `legit < threshold` flag as `flagged`.
- If `duplicate_of` present, set `status = duplicate` and link.

Response: 200 OK.

Feature H — Trending & Search

GET `/api/v1/trending?region=...&limit=10`

Return curated list of trending problem report summaries.

Response:

```
{
  "region": "Delhi",
  "trending": [
    { "report_id": "...", "title": "Garbage near market", "score": 87, "upvotes": 500 }
  ]
}
```

Algorithm:

- Score = weighted function of recency, upvotes, comments, resolved status, images present.
 - Use Redis sorted sets for real-time ranking.
-

GET `/api/v1/search?q=power outage®ion=Delhi&category=...`

Full-text search (ElasticSearch) returns paged list.

Feature I — Rewards & Redemption

GET `/api/v1/rewards`

List all rewards, their required points and availability.

POST `/api/v1/rewards/:id/redeem`

Auth required. Request:

```
{ "reward_id": "reward-uuid", "delivery_address": { ... }, "proof": "optional" }
```

Validations:

- User has enough points
- `max_per_user` not exceeded

Processing:

- Deduct points (transaction)
- Create `Redemption` with `status: requested`
- Notify admin for approval

Response: redemption object.

Feature J — Notifications

GET `/api/v1/notifications`

Paged list.

POST `/api/v1/notifications/subscribe`

Subscribe to push or email notifications.

Server: push via FCM or APNs for mobile, email via SES/Sendgrid.

Feature K — Audit & Appeals

GET `/api/v1/audit/reports/:id`

Admin-only: returns audit logs for a report (flags, edits, assignments).

POST `/api/v1/reports/:id/appeal`

User appeal to request unban/unflag.

4) Background jobs, AI & image pipeline

Jobs & Queues

- **Image processing queue:** generate thumbnails, compress, store CDN URLs, detect EXIF tampering.
- **AI analysis queue:** send images to model(s) — classification (garbage), severity, duplicate detection via perceptual hashing + nearest neighbor search.
- **Duplicate detection job:** compute pHash/perceptual hash and query nearest matching reports in geo-radius (e.g., within 200 meters & last 30 days).
- **Notifications job:** push when MCD resolves, when trending in user's area.
- **Points awarding job:** When report resolved, run points award transaction (idempotent).
- **Trending aggregator:** continuous job updating Redis sorted sets for trending per region.

AI specifics

- Use model endpoint (TensorFlow/PyTorch) returning:
 - `legit_probability`, `category_confidence`, `severity_estimate`, `tags` (organic/metal/plastic).
- Duplicate detection: combine image hash + location proximity + text similarity.
- AI responses stored in `report.ai_score`.

Idempotency

- POST `/api/v1/reports` should accept `client_idempotency_key` to avoid duplicate creation on resubmits.
-

5) Security, Rate limits, Privacy

Authentication & Authorization

- Use JWT with standard claims.
- Role-based access: `user`, `admin`, `mcd`.
- Admin tokens rotate, require 2FA onboarding.

Rate limiting

- Unauthenticated: 30 requests / minute.
- Authenticated: 120 requests / minute.
- Heavier endpoints (create report): 10 reports / day per user (increase with trust).
- Use Redis rate limiter & IP-based throttling.

Abuse prevention

- Trust score: block or soft-limit users with low trust.
- Device fingerprinting + phone verification to curb ban evasion.
- CAPTCHA for suspicious flows.

Privacy & Data access

- Exact GPS stored but only accessible to MCD/admin with region scope.
- Public API returns masked area name (ward/sector) not coordinates.
- GDPR/India privacy compliance: data retention policy; implement delete requests for user data, anonymization after a retention window unless required for legal reasons.
- Aadhaar: strongly avoid storing raw Aadhaar. If required for verification, integrate with government APIs using secure, compliant method and store

proof only.

6) Monitoring & Operational recommendations

- **Metrics:**
 - Reports created / day, reports resolved / day, average time to resolution.
 - Trending queue size, AI queue latency, image processing times.
 - Error rates per endpoint.
 - **Logging:**
 - Structured logs, redact PII, centralize to ELK or Datadog.
 - **Tracing:**
 - Use distributed tracing to track request → AI job → DB update flows.
 - **SLOs:**
 - API latency p95 < 300ms for read endpoints.
 - Report creation p95 < 2s (excluding image upload).
 - **Backups:**
 - DB backups daily, snapshot retention 30 days.
 - **CDN** for media with signed URLs.
-

7) Additional improvements & recommended features (priority ranked)

Immediate (High)

1. **Presigned image uploads** (S3/GCS) to avoid server bandwidth.
2. **AI advisory (not authoritative)** flagging on UI only until robust.
3. **Idempotent create with client tokens.**
4. **Admin audit logs** for all MCD actions.
5. **Geo-indexing** for queries and trending.

Mid-term (Medium)

1. **Reputation model** that weights votes if from verified/trustworthy users.
2. **Offline queue / resume** on client with server-side idempotency support.
3. **Auto-merge duplicate reports** (manual approval) with logic: same image hash + location/time threshold.
4. **Reward redemption flow** integrations with partner vendors; generate voucher codes.

Long-term (Nice-to-have)

1. **Computer vision: before/after verify** (compare pre- and post-clean images to auto-approve resolution).
 2. **Predictive hotspots** for proactive clean-up using spatial-temporal ML.
 3. **Public dashboards for city authorities** with analytics & export.
 4. **Multi-language support** for UI content & moderation.
-

8) Example sequence: Report lifecycle & required APIs

1. **User captures image & requests presigned URL**
 - GET `/api/v1/uploads/presign?filename=...&content_type=image/jpeg` → Response presigned URL.
 2. **Client uploads image to S3 using presigned URL** (client → S3).
 3. **Client POSTs report** with image URL(s) → `/api/v1/reports` (server stores, returns id).
 4. **Server enqueues AI job**; AI updates via `/internal/ai/reports/:id/result`.
 5. **Community votes/comments** via `/api/v1/reports/:id/vote` and `/comments`.
 6. **Admin views, assigns, resolves** using admin endpoints.
 7. **On resolve**, server awards points (transactional) and creates `Redemption` eligibility if thresholds met.
-

9) Example JSON responses & errors

Standard error format

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid email format",
    "fields": { "email": "must be a valid email address" }
  }
}
```

401 Unauthorized

```
{ "error": { "code": "UNAUTHORIZED", "message": "Invalid or expired token" } }
```

429 Rate limit

```
{ "error": { "code": "RATE_LIMIT", "message": "Too many requests. Retry after 60s" } }
```

10) DB / Index recommendations & capacity planning

- Use geospatial index for `Report.location` (PostGIS or Mongo2dSphere).
- Create compound index `(category, status, created_at)` for fast feed queries.
- Use Redis for caching `trending` per region (sorted sets) and rate-limits.
- Media store: S3 + CloudFront (CDN).
- Use message broker (RabbitMQ / SQS / PubSub) for queues: AI jobs, image processing, notifications.

Scale guidance:

- Start with single region; plan sharding by region if > 1M reports.
 - Use worker autoscaling for AI jobs; GPUs for CV models.
-

11) Final notes on validations & transactional behavior

- Use DB transactions for critical actions (awarding points + marking resolved + audit log + redemption ledger).
- All write endpoints should be idempotent or accept idempotency keys for retries.
- Validate client-sent `area_name` against service region map to prevent fake geo tags; otherwise check distance between lat/lng and declared area center.